

Database Principles: Fundamentals of Design, Implementation, and Management

Lecture 2: Data Models

Updated: September 2025

Learning Outcomes

After completing this lecture, students will be able to:

- Explain what **data modeling** is and why it is important.
- Identify the basic building blocks of **data models**.
- Outline the **evolution** of major data models.
- Describe the **logical view of data** in a relational data model.
- Identify and explain **different types of keys** used in relational databases.
- Explain the importance of **integrity rules** in maintaining data accuracy and consistency.
- Describe the role and purpose of a **data dictionary** in a database system.
- Explain how **relationships** are established and maintained within a relational database.

Introduction

- Designers, programmers, and end users **see data in different ways**
- **Different views of same data** lead to designs that do not reflect organization's operation
- Data modeling **reduces complexities** of database design
- Various degrees of **data abstraction** help reconcile varying views of same data

Data Modeling and Data Models

- **Data models**
 - Relatively **simple representations** of complex real-world data structures
 - Often **graphical**
- **Model:** **an abstraction** of a real-world object or event
 - Useful in **understanding complexities** of the real-world environment
- **Data modeling** is **iterative and progressive**

Data Modeling and Data Models (cont'd.)

The Importance of Data Models:

- **Facilitate interaction** among the designer, the applications programmer, and the end user
- End users have **different views and needs** for data
- Data model **organizes data** for various users
- Data model is **an abstraction**:
 - A data model itself **does not contain actual data**: it only defines the structure and rules for how data is organized.
 - You cannot retrieve or query data directly from a model, because it's **just a blueprint or design**, not a database.

Data Modeling and Data Models (cont'd.)

Data Model Basic Building Blocks:

- **Entity:** anything about which data are to be collected and stored
- **Attribute:** a characteristic of an entity
- **Relationship:** describes an association among entities
 - One-to-many (1:M) relationship
 - Many-to-many (M:N or M:M) relationship
 - One-to-one (1:1) relationship
- **Constraint:** a restriction placed on the data

The Evolution of Data Models

**TABLE
3.1**

Evolution of Major Data Models

GENERATION	TIME	DATA MODEL	EXAMPLES	COMMENTS
First	1960s–1970s	File system	VMS/VSAM	Used mainly on IBM mainframe systems Managed records, not relationships
Second	1970s	Hierarchical and network	IMS, ADABAS, IDS-II	Early database systems Navigational access
Third	Mid-1970s	Relational	DB2 Oracle MS SQL Server MySQL	Conceptual simplicity Entity relationship (ER) modeling and support for relational data modeling
Fourth	Mid-1980s	Object-oriented Object/ relational (O/R)	Versant Objectivity/DB DB2 UDB Oracle 11g	Object/relational supports object data types Star Schema support for data warehousing Web databases become common
Fifth	Mid-1990s	XML Hybrid DBMS	dbXML Tamino DB2 UDB Oracle 11g MS SQL Server	Unstructured data support O/R model supports XML documents Hybrid DBMS adds object front end to relational databases Support large databases (terabyte size)
Emerging Models: NoSQL	Late 2000s to present	Key-value store Column store	SimpleDB (Amazon) BigTable (Google) Cassandra (Apache)	Distributed, highly scalable High performance, fault tolerant Very large storage (petabytes) Suited for sparse data Proprietary API

Hierarchical and Network Models

- **The hierarchical model**

- Developed in the 1960s to manage large amounts of data for manufacturing projects
- Basic logical structure is represented by an upside-down “tree”
- Structure contains levels or segments

- **Network model**

- Created to represent complex data relationships more effectively than the hierarchical model
- Improves database performance
- Imposes a database standard
- Resembles hierarchical model
 - Record may have more than one parent

The Relational Model

- Developed by E.F. Codd (IBM) in 1970
- Table (relations)
 - Matrix consisting of row/column intersections
 - Each row in a relation is called a tuple
- Relational models were considered impractical in 1970
- Model was conceptually simple at expense of computer overhead
- **Relational data management system (RDBMS)**
 - Performs same functions provided by hierarchical model
 - Hides complexity from the user
- **Relational diagram**
 - Representation of entities, attributes, and relationships
- Relational table stores collection of related entities

**FIGURE
3.1**

Linking relational tables

Table name: AGENT (first six attributes)

Database name: Ch03_InsureCo

AGENT_CODE	AGENT_LNAME	AGENT_FNAME	AGENT_INITIAL	AGENT_AREACODE	AGENT_PHONE
501	Alby	Alex	B	713	226-1248
502	Hahn	Leah	F	615	862-1244
503	Okon	John	T	615	123-5589

Link through AGENT_CODE

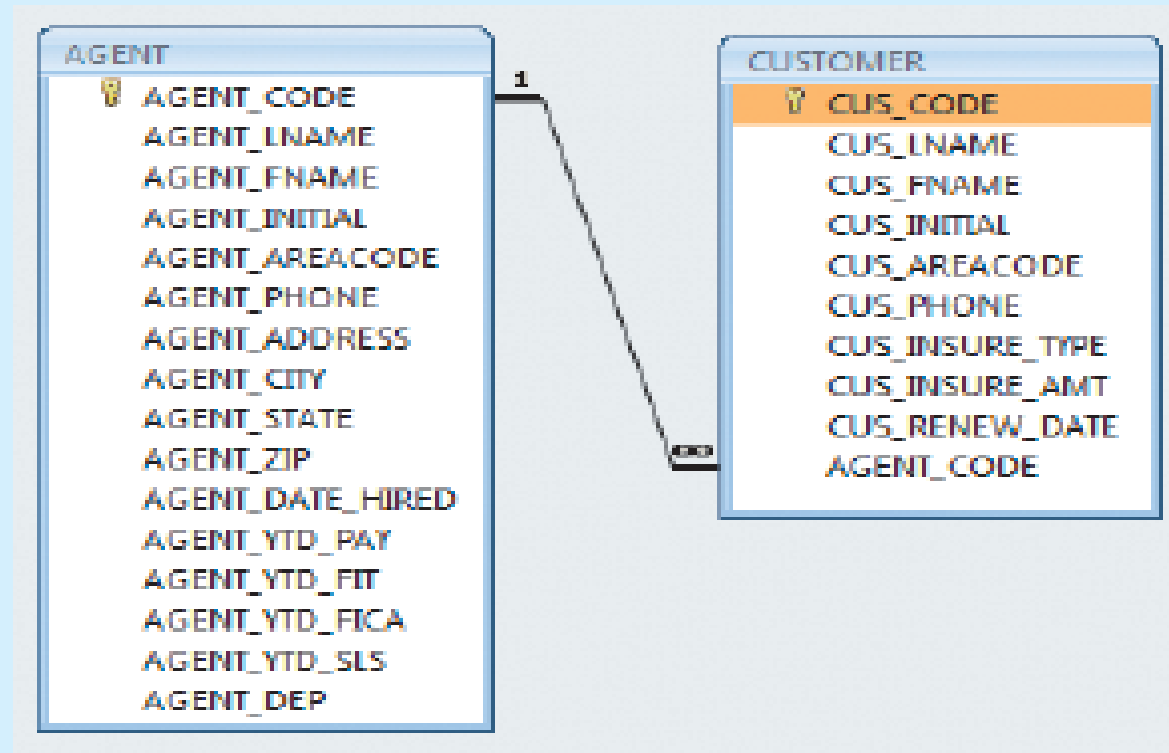
Table name: CUSTOMER

CUS_CODE	CUS_LNAME	CUS_FNAME	CUS_INITIAL	CUS_AREACODE	CUS_PHONE	CUS_INSURE_TYPE	CUS_INSURE_AMT	CUS_RENEW_DATE	AGENT_CODE
10010	Ramas	Alfred	A	615	844-2573	T1	100.00	05-Apr-2012	502
10011	Dunne	Leona	K	713	894-1238	T1	250.00	16-Jun-2012	501
10012	Smith	Kathy	W	615	894-2285	S2	150.00	29-Jan-2013	502
10013	Olowski	Paul	F	615	894-2180	S1	300.00	14-Oct-2012	502
10014	Orlando	Myron		615	222-1672	T1	100.00	28-Dec-2013	501
10015	O'Brian	Amy	B	713	442-3381	T2	950.00	22-Sep-2012	503
10016	Brown	James	G	615	297-1228	S1	120.00	25-Mar-2013	502
10017	Williams	George		615	290-2558	S1	250.00	17-Jul-2012	503
10018	Farriss	Anne	O	713	382-7185	T2	100.00	03-Dec-2012	501
10019	Smith	Olette	K	615	297-3809	S2	500.00	14-Mar-2013	503

SOURCE: Course Technology/Cengage Learning

**FIGURE
3.2**

A relational diagram



SOURCE: Course Technology/Cengage Learning

The Entity Relationship Model

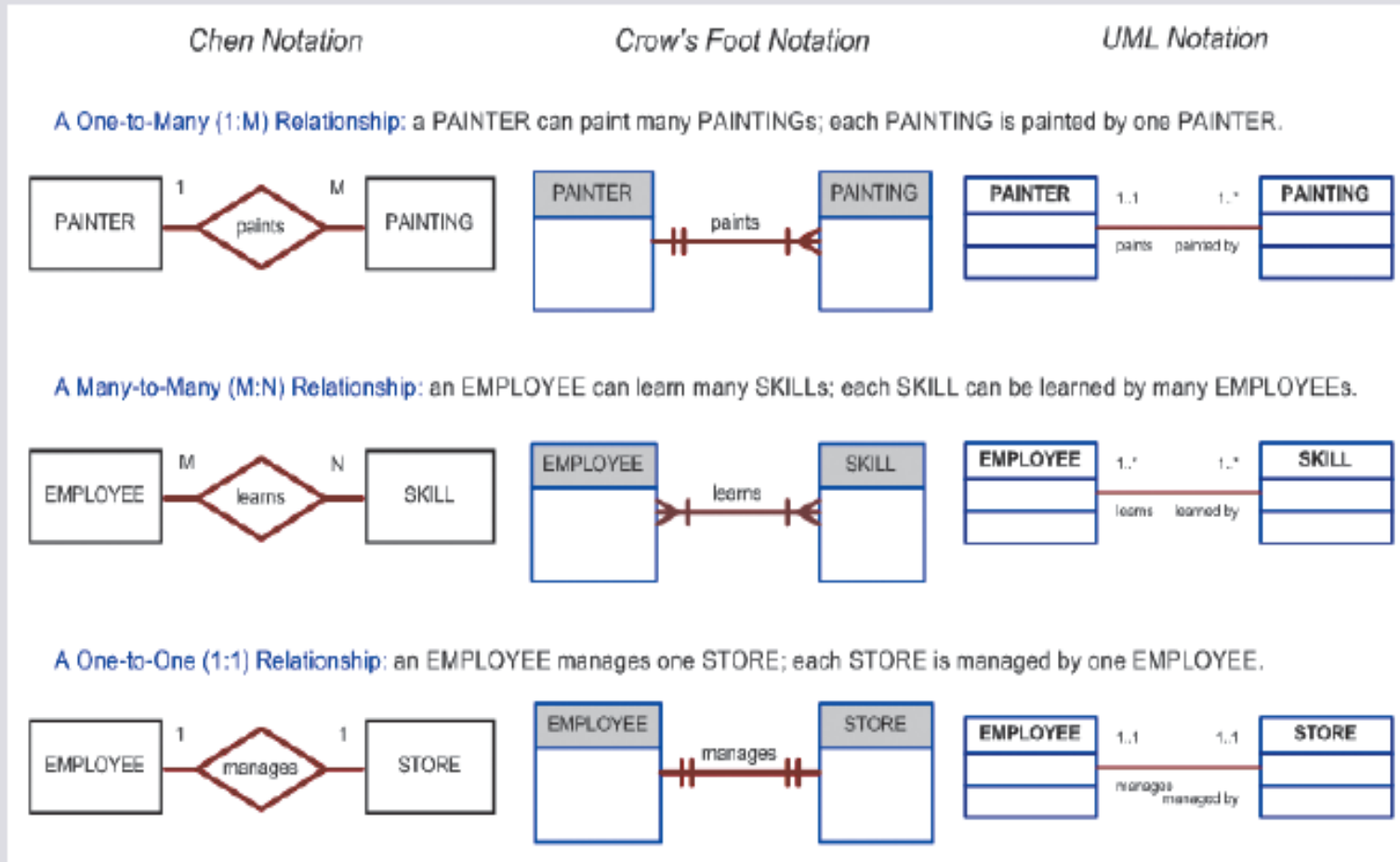
- Widely accepted standard for data modeling
- Introduced by Chen in 1976
- Graphical representation of entities and their relationships in a database structure
- **Entity relationship diagram (ERD)**
 - Uses graphic representations to model database components
 - Entity is mapped to a relational table

The Entity Relationship Model (cont'd.)

- **Entity instance** (or occurrence) is row in table
- **Entity set** is collection of like entities
- **Connectivity** labels types of relationships
- Relationships are expressed using **Chen notation**
 - Relationships are **represented by a diamond**
 - Relationship name is written **inside the diamond**
- **Crow's Foot notation** used as design standard in this book

FIGURE 3.3

The ER model notations



SOURCE: Course Technology/Cengage Learning

The Object-Oriented (OO) Model

- **Data and relationships** are contained in a single structure known as an **object**
- OODM (object-oriented data model) is the **basis for OODBMS**
 - Semantic data model
- **An object:**
 - Contains **operations**
 - Are **self-contained**: a basic building-block for autonomous structures
 - Is an **abstraction** of a real-world entity

The Object-Oriented (OO) Model (cont'd.)

- Attributes describe the **properties of an object**
- Objects that share similar characteristics are **grouped in classes**
- Classes are organized in a **class hierarchy**
- **Inheritance**: object inherits methods and attributes of parent class
- UML **based on OO concepts** that describe diagrams and symbols
 - Used to **graphically model a system**

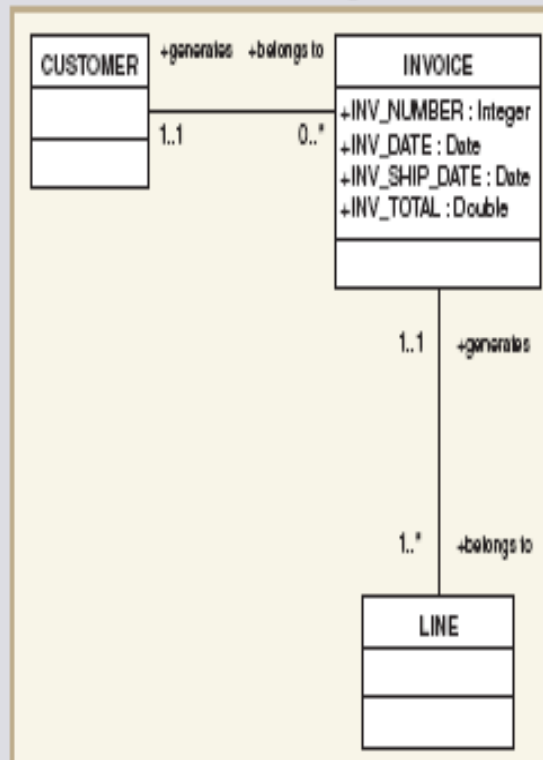
**FIGURE
3.4**

A comparison of OO, UML, and ER models

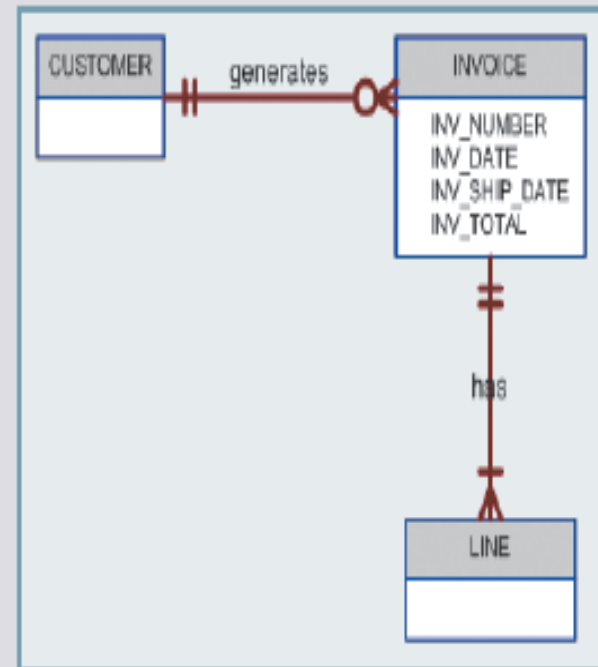
Object Representation



UML Class Diagram



ER Model



SOURCE: Course Technology/Cengage Learning

Object/Relational and XML

- **Extended relational data model (ERDM)**
 - Semantic data model developed in response to **increasing complexity of applications**
 - Includes **many of OO model's best features**
 - Often described as an **object/relational database management system (O/RDBMS)**
- The Internet revolution created the potential to exchange **critical business information**
- In this environment, **Extensible Markup Language (XML)** emerged as the de facto standard
- Current databases **support XML**
 - XML: the **standard protocol** for data exchange among systems and Internet services

Emerging Data Models: Big Data and NoSQL

- **Big Data**

- Find new and better ways to **manage large amounts of Web-generated data** and derive business insight from it
- Simultaneously provides **high performance and scalability** at a reasonable cost
- Relational approach **does not always match the needs** of organizations with Big Data challenges

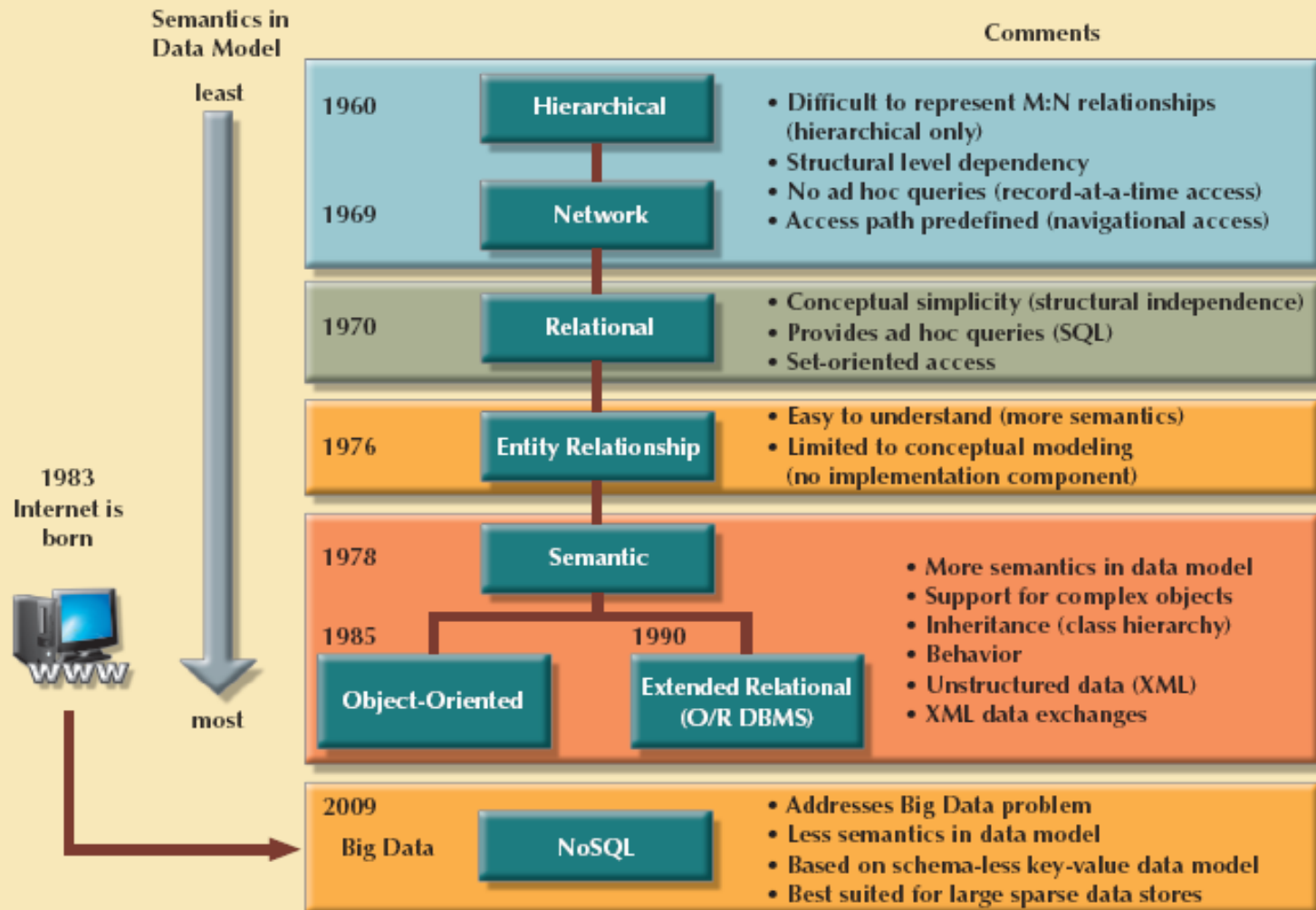
Emerging Data Models: Big Data and NoSQL (cont'd.)

- **NoSQL databases**

- Not based on the relational model, hence the name NoSQL
- Supports distributed database architectures
- Provides high scalability, high availability, and fault tolerance
- Supports very large amounts of sparse data
- Geared toward performance rather than transaction consistency

**FIGURE
3.6**

The evolution of data models



SOURCE: Course Technology/Cengage Learning

A Logical View of Data

- **Relational model**
 - View data logically rather than physically
- **Table**
 - Structural and data independence
 - Resembles a file conceptually
- Relational database model is easier to understand than hierarchical and network models
- **Tables and Their Characteristics**
 - Logical view of relational database is based on relation: relation thought of as a table
 - Table: two-dimensional structure composed of rows and columns: persistent representation of logical relation
 - Contains group of related entities (entity set)

TABLE
4.1

Characteristics of a Relational Table

1	A table is perceived as a two-dimensional structure composed of rows and columns.
2	Each table row (tuple) represents a single entity occurrence within the entity set.
3	Each table column represents an attribute, and each column has a distinct name.
4	Each intersection of a row and column represents a single data value.
5	All values in a column must conform to the same data format.
6	Each column has a specific range of values known as the attribute domain .
7	The order of the rows and columns is immaterial to the DBMS.
8	Each table must have an attribute or combination of attributes that uniquely identifies each row.

FIGURE 4.1 STUDENT table attribute values

Table name: STUDENT

Database name: Ch04_TinyCollege

STU_NUM	STU_LNAME	STU_FNAME	STU_INIT	STU_DOB	STU_HRS	STU_CLASS	STU_GPA	STU_TRANSFER	DEPT_CODE	STU_PHONE	PROF_NUM
321452	Bowser	William	C	12-Feb-1975	42	So	2.84	No	BIOL	2134	205
324257	Smithson	Anne	K	15-Nov-1981	81	Jr	3.27	Yes	CIS	2256	222
324258	Brewer	Juliette		23-Aug-1969	36	So	2.26	Yes	ADCT	2256	228
324269	Oblonski	Walter	H	16-Sep-1976	66	Jr	3.09	No	CIS	2114	222
324273	Smith	John	D	30-Dec-1958	102	Sr	2.11	Yes	ENGL	2231	199
324274	Katinga	Raphael	P	21-Oct-1979	114	Sr	3.15	No	ADCT	2267	228
324291	Robertson	Gerald	T	08-Apr-1973	120	Sr	3.87	No	EDU	2267	311
324299	Smith	John	B	30-Nov-1986	15	Fr	2.92	No	ADCT	2315	230

STU_NUM = Student number
STU_LNAME = Student last name
STU_FNAME = Student first name
STU_INIT = Student middle initial
STU_DOB = Student date of birth
STU_HRS = Credit hours earned
STU_CLASS = Student classification
STU_GPA = Grade point average
STU_TRANSFER = Student transferred from another institution
DEPT_CODE = Department code
STU_PHONE = 4-digit campus phone extension
PROF_NUM = Number of the professor who is the student's advisor

SOURCE: Course Technology/Cengage Learning

Keys

- Each row in a table must be **uniquely identifiable**
- **Key**: one or more attributes that **determine other attributes**
 - Key's role is **based on determination**
 - If you know the value of attribute A, you can determine the value of attribute B
 - **Functional dependence**
 - Attribute B is functionally dependent on A if all rows in table that agree in value for A also agree in value for B

Keys (cont'd.)

Types of Keys:

1.Key attribute

- A **key attribute** is an **attribute (column)** in a table that helps **uniquely identify a record (row)** within that table.
- It is part of or can serve as a **primary key**.
- It contains **unique values** — no two rows can have the same key attribute value.
- It ensures **data integrity** and supports **relationships** between tables.
- **Example:**
 - **StudentID** is the **key attribute** because it uniquely identifies each student record.

Keys (cont'd.)

2. Candidate key

- A **candidate key** is an **attribute or a set of attributes** that can **uniquely identify each record** in a table.
- Each candidate key has the potential to become the **primary key**.
- A table can have **multiple candidate keys**, but only **one** is chosen as the **primary key**.
- All candidate keys must contain **unique** and **non-null** values.
- **Example:**
 - **StudentID** and **ICNumber** can both uniquely identify a student.
 - Therefore, both are **candidate keys**, but only one (for example, **StudentID**) is chosen as the **primary key**.

Keys (cont'd.)

3. Primary key

- A **primary key** is an **attribute or set of attributes** that **uniquely identifies each record** in a table.
- Each table can have **only one primary key**.
- The primary key value must be **unique** for every record.
- It **cannot contain NULL values**.
- It is used to **enforce entity integrity** and to **link related tables** through foreign keys.
- **Example:**
 - **StudentID** is the **primary key** because it uniquely identifies each student record.

Keys (cont'd.)

4. Alternate key

- An **alternate key** (also called an **alternative key**) is a **candidate key** that is not chosen as the **primary key**.
- A table can have **multiple candidate keys**, but only **one** becomes the **primary key**.
- The remaining candidate keys are known as **alternate keys**.
- Alternate keys can still be used to **uniquely identify** records if needed.
- **Example:**
 - **StudentID** and **ICNumber** are both **candidate keys**.
 - If **StudentID** is selected as the **primary key**, then **ICNumber** becomes the **alternate key**.

Keys (cont'd.)

5. Composite key

- A **composite key** is a **primary key** that consists of **two or more attributes (columns)** used together to uniquely identify a record in a table.
- A single attribute alone is **not sufficient** to uniquely identify a row.
- The **combination** of the attributes ensures **uniqueness**.
- Commonly used when no single column has unique values.
- **Example:**
 - In a table **Student_Course**, **StudentID** alone is not unique, **CourseID** alone is not unique.
 - But the **combination (StudentID, CourseID)** uniquely identifies each record — this is the **composite key**.

Keys (cont'd.)

6. Foreign key

- A **foreign key** is an **attribute (or set of attributes)** in one table that **links to the primary key** in another table.
- It is used to **create relationships** between two tables.
- It ensures **referential integrity** — meaning the value in the foreign key must **exist in the referenced (parent) table's primary key**.
- A table can have **multiple foreign keys**.
- **StudentID** in the **Enrollment** table is the **foreign key**.
- It refers to **StudentID** in the *Student* table (the **primary key**).
- This relationship ensures that **only existing students** can appear in the Enrollment table.

Keys (cont'd.)

- **Controlled redundancy**
 - Makes the relational database work
 - Tables within the database share common attributes
 - Enables tables to be linked together
 - Multiple occurrences of values not redundant when required to make the relationship work
 - Redundancy exists only when there is unnecessary duplication of attribute values

**FIGURE
4.2**

An example of a simple relational database

Table name: **PRODUCT**

Primary key: **PROD_CODE**

Foreign key: **VEND_CODE**

PROD_CODE	PROD_DESCRIPT	PROD_PRICE	PROD_ON_HAND	VEND_CODE
001278-AB	Claw hammer	12.95	23	232
123-21UUY	Houselite chain saw, 16-in. bar	189.99	4	235
QER-34256	Sledge hammer, 16-lb. head	18.63	6	231
SRE-657UG	Rat-tail file	2.99	15	232
ZZX/3245Q	Steel tape, 12-ft. length	6.79	8	235

Database name: **Ch04_SaleCo**

link

Table name: **VENDOR**

Primary key: **VEND_CODE**

Foreign key: none

VEND_CODE	VEND_CONTACT	VEND_AREACODE	VEND_PHONE
230	Shelly K. Smithson	608	555-1234
231	James Johnson	615	123-4536
232	Annelise Crystall	608	224-2134
233	Candice Wallace	904	342-6567
234	Arthur Jones	615	123-3324
235	Henry Ortozo	615	899-3425

SOURCE: Course Technology/Cengage Learning

Integrity Rules

- Many RDBMs **enforce integrity rules** automatically
- Safer to ensure that application design conforms to **entity and referential integrity rules**
- **Entity integrity**
 - Each row (entity instance) in the table has its **own unique identity**
- **Referential integrity**
 - FK contains a value that **refers to an existing valid tuple (row)** in another relation

**FIGURE
4.3**

An illustration of integrity rules

Table name: CUSTOMER

Database name: Ch04_InsureCo

Primary key: CUS_CODE

Foreign key: AGENT_CODE

CUS_CODE	CUS_LNAME	CUS_FNAME	CUS_INITIAL	CUS_RENEW_DATE	AGENT_CODE
10010	Ramas	Alfred	A	05-Apr-2012	502
10011	Dunne	Leona	K	16-Jun-2012	501
10012	Smith	Kathy	W	29-Jan-2013	502
10013	Olowski	Paul	F	14-Oct-2012	
10014	Orlando	Myron		28-Dec-2012	501
10015	O'Brian	Amy	B	22-Sep-2012	503
10016	Brown	James	G	25-Mar-2013	502
10017	Williams	George		17-Jul-2012	503
10018	Farriss	Anne	G	03-Dec-2012	501
10019	Smith	Olette	K	14-Mar-2013	503

Table name: AGENT (only five selected fields are shown)

Primary key: AGENT_CODE

Foreign key: none

AGENT_CODE	AGENT_AREACODE	AGENT_PHONE	AGENT_LNAME	AGENT_YTD_SLS
501	713	228-1249	Alby	132735.75
502	615	882-1244	Hahn	138967.35
503	615	123-5589	Okon	127093.45

SOURCE: Course Technology/Cengage Learning

Integrity Rules (cont'd.)

- **Nulls**

- No data entry
- Not permitted in primary key
- Should be avoided in other attributes
- Can represent:
 - An unknown attribute value
 - A known, but missing, attribute value
 - A “not applicable” condition
- Can create problems when functions such as COUNT, AVERAGE, and SUM are used
- Can create logical problems when relational tables are linked
- Designers use flags to avoid nulls
- Flags indicate absence of some value

**TABLE
4.4**

Integrity Rules

ENTITY INTEGRITY	DESCRIPTION
Requirement	All primary key entries are unique, and no part of a primary key may be null.
Purpose	Each row will have a unique identity, and foreign key values can properly reference primary key values.
Example	No invoice can have a duplicate number, nor can it be null. In short, all invoices are uniquely identified by their invoice number.
REFERENTIAL INTEGRITY	DESCRIPTION
Requirement	A foreign key may have either a null entry, as long as it is not a part of its table's primary key, or an entry that matches the primary key value in a table to which it is related. (Every non-null foreign key value <i>must</i> reference an <i>existing</i> primary key value.)
Purpose	It is possible for an attribute <i>not</i> to have a corresponding value, but it will be impossible to have an invalid entry. The enforcement of the referential integrity rule makes it impossible to delete a row in one table whose primary key has mandatory matching foreign key values in another table.
Example	A customer might not yet have an assigned sales representative (number), but it will be impossible to have an invalid sales representative (number).

**TABLE
4.5**

A Dummy Variable Value Used as a Flag

AGENT_CODE	AGENT_AREACODE	AGENT_PHONE	AGENT_LNAME	AGENT_YTD_SLS
-99	000	000-0000	None	\$0.00

The Data Dictionary and System Catalog

- **Data dictionary**

- Provides **detailed** accounting of all tables found within the user/designer-created database
- Contains **(at least) all the attribute names and characteristics** for each table in the system
- Contains **metadata**: data about data

- **System catalog**

- Contains **metadata**
- **Detailed system data dictionary** that describes all objects within the database

TABLE
4.6

A Sample Data Dictionary

TABLE NAME	ATTRIBUTE NAME	CONTENTS	TYPE	FORMAT	RANGE	REQUIRED	PK or FK	FK REFERENCED TABLE
CUSTOMER	CUS_CODE	Customer account code	CHAR(5)	99999	10000–99999	Y	PK FK	AGENT_CODE
	CUS_LNAME	Customer last name	VARCHAR(20)	Xxxxxxxxxx		Y		
	CUS_FNAME	Customer first name	VARCHAR(20)	Xxxxxxxxxx		Y		
	CUS_INITIAL	Customer initial	CHAR(1)	X				
	CUS_RENEW_DATE	Customer insurance renewal date	DATE	dd-mmm-yyyy				
	AGENT_CODE	Agent code	CHAR(3)	999				
AGENT	AGENT_CODE	Agent code	CHAR(3)	999		Y	PK	
	AGENT_AREACODE	Agent area code	CHAR(3)	999		Y		
	AGENT_PHONE	Agent telephone number	CHAR(8)	999-9999		Y		
	AGENT_LNAME	Agent last name	VARCHAR(20)	Xxxxxxxxxx		Y		
	AGENT_YTD_SLS	Agent year-to-date sales	NUMBER(9,2)	9,999,999.99				

FK =Foreign key

PK =Primary key

CHAR =Fixed character length data (1–255 characters)

VARCHAR =Variable character length data (1–2,000 characters)

NUMBER =Numeric data (NUMBER(9,2)) are used to specify numbers with two decimal places and up to nine digits, including the decimal places. Some RDBMSs permit the use of a MONEY or CURRENCY data type.

Relationships within the Relational Database

- **Relationships** within the relational database describe how tables are connected to each other through keys (usually primary and foreign keys).
- These relationships ensure that data is linked logically and can be combined or referenced across tables.
- **1:M relationship**
 - Relational modeling ideal
 - Should be the norm in any relational database design
- **1:1 relationship**
 - Should be rare in any relational database design
- **M:N relationships**
 - Cannot be implemented as such in the relational model
 - M:N relationships can be changed into 1:M relationships

The 1:M Relationship

- Relational database **norm**
- **Found in any** database environment

**FIGURE
4.17**

**The 1:M relationship between
PAINTER and PAINTING**



SOURCE: Course Technology/Cengage Learning

**FIGURE
4.18**

The implemented 1:M relationship between PAINTER and PAINTING

Table name: PAINTER

Primary key: PAINTER_NUM

Foreign key: none

Database name: Ch04_Museum

PAINTER_NUM	PAINTER_LNAME	PAINTER_FNAME	PAINTER_INITIAL
123	Ross	Georgette	P
126	Itero	Julio	G

Table name: PAINTING

Primary key: PAINTING_NUM

Foreign key: PAINTER_NUM

PAINTING_NUM	PAINTING_TITLE	PAINTER_NUM
1338	Dawn Thunder	123
1339	Vanilla Roses To Nowhere	123
1340	Tired Flounders	126
1341	Hasty Exit	123
1342	Plastic Paradise	126

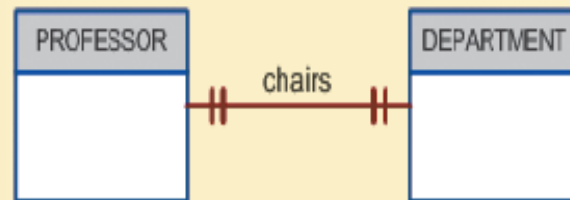
SOURCE: Course Technology/Cengage Learning

The 1:1 Relationship

- One entity related to only one other entity, and vice versa
- Sometimes means that entity components were **not defined properly**
- Could indicate that two entities actually **belong in the same table**
- **Certain conditions** absolutely require their use

FIGURE
4.21

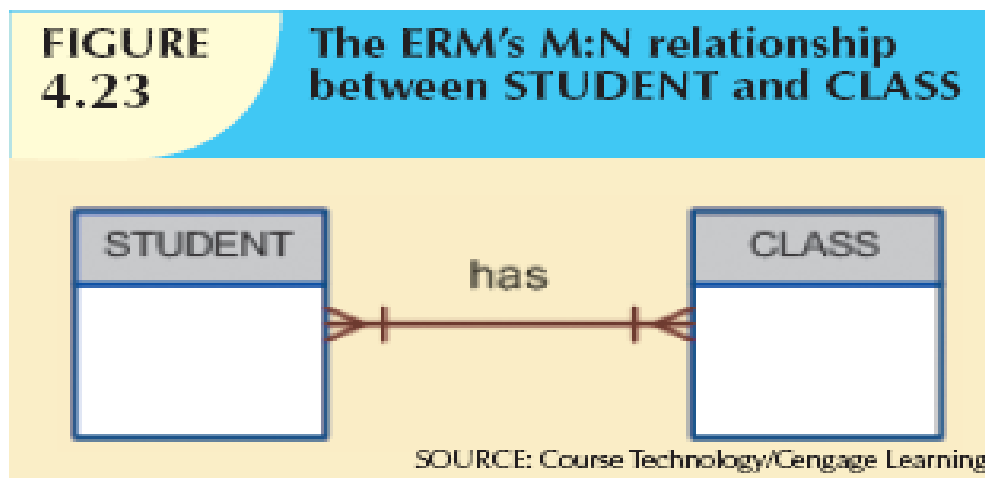
The 1:1 relationship between PROFESSOR and DEPARTMENT



SOURCE: Course Technology/Cengage Learning

The M:N Relationship

- Implemented by **breaking it up** to produce a set of 1:M relationships
- Avoid problems inherent to M:N relationship by **creating a composite entity**
 - Includes as **foreign keys the primary keys** of tables to be linked



**FIGURE
4.25**

Converting the M:N relationship into two 1:M relationships

Table name: STUDENT
Primary key: STU_NUM
Foreign key: none

Database name: Ch04_CollegeTry2

STU_NUM	STU_LNAME
321452	Bowser
324257	Smithson

Table name: ENROLL
Primary key: CLASS_CODE + STU_NUM
Foreign key: CLASS_CODE, STU_NUM

CLASS_CODE	STU_NUM	ENROLL_GRADE
10014	321452	C
10014	324257	B
10018	321452	A
10018	324257	B
10021	321452	C
10021	324257	C

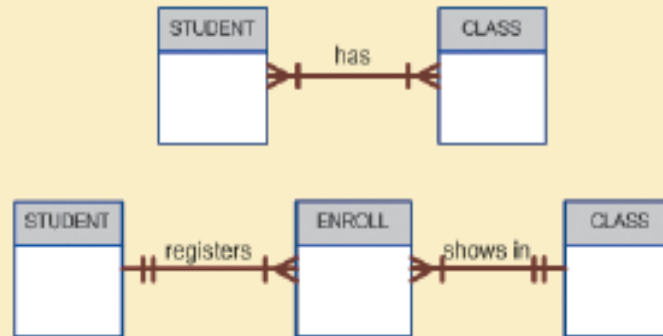
Table name: CLASS
Primary key: CLASS_CODE
Foreign key: CRS_CODE

CLASS_CODE	CRS_CODE	CLASS_SECTION	CLASS_TIME	CLASS_ROOM	PROF_NUM
10014	AOCT-211	3	TTh 2:30-3:45 p.m.	BUS252	342
10018	CIS-220	2	MW 9:00-9:50 a.m.	KLR211	114
10021	QM-261	1	MW 8:00-8:50 a.m.	KLR200	114

SOURCE: Course Technology/Cengage Learning

**FIGURE
4.26**

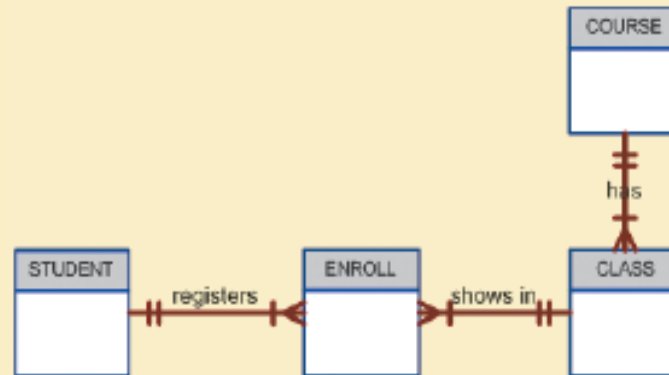
Changing the M:N relationship to two 1:M relationships



SOURCE: Course Technology/Cengage Learning

**FIGURE
4.27**

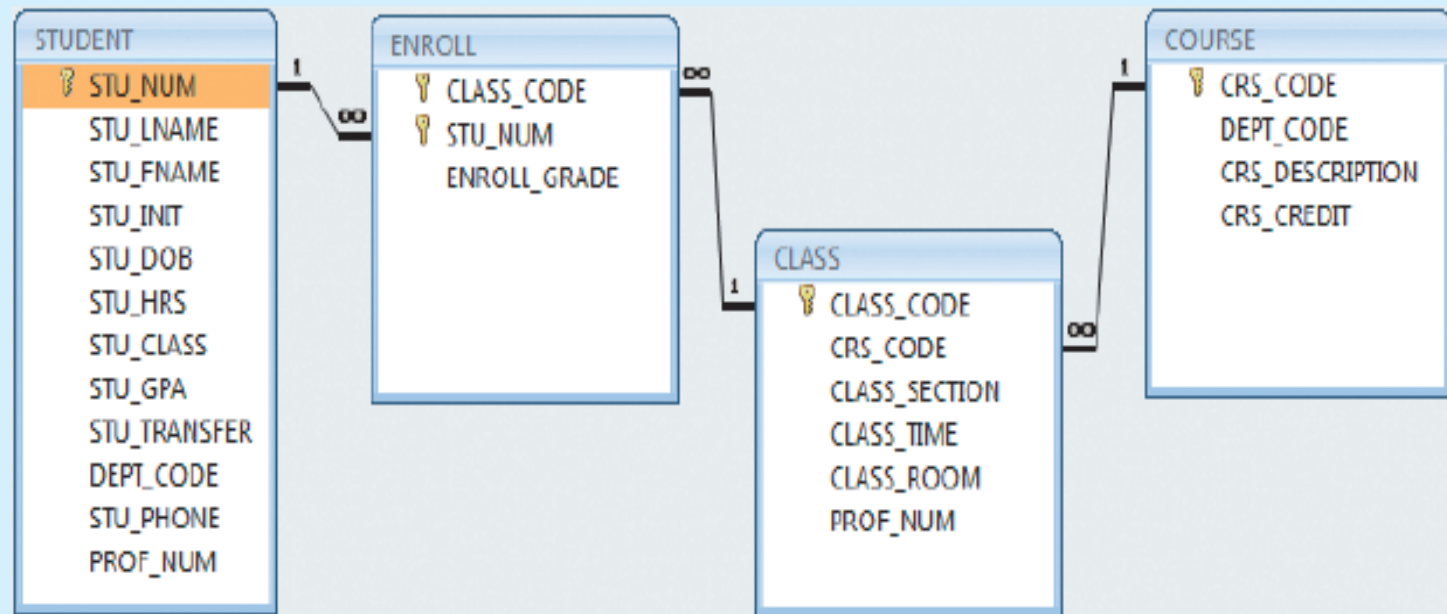
The expanded entity relationship model



SOURCE: Course Technology/Cengage Learning

**FIGURE
4.28**

The relational diagram for the Ch04_TinyCollege database



SOURCE: Course Technology/Cengage Learning

Summary

- A data model is an abstraction of a complex real-world data environment
- Basic data modeling components:
 - Entities
 - Attributes
 - Relationships
 - Constraints
- Business rules identify and define basic modeling components

Summary (cont'd.)

- Data model **common characteristics**:
 - Conceptual simplicity with semantic completeness
 - Represent the real world as closely as possible
 - Real-world transformations must comply with consistency and integrity characteristics
- Each new data model **capitalized on the shortcomings of previous models**
- Some models **better suited for some tasks**

Summary (cont'd.)

- **Types of Keys**

- Candidate Key – possible unique identifiers
- Primary Key – uniquely identifies a record
- Alternate Key – candidate key not chosen as primary
- Composite Key – made of multiple attributes
- Foreign Key – links tables together

- **Integrity Rules**

- Entity Integrity – primary key must be unique and not null
- Referential Integrity – foreign key values must exist in the parent table
- Maintains data accuracy and consistency

Summary (cont'd.)

- **Data Dictionary**
 - A repository that stores details about database objects (tables, fields, data types).
 - Used by designers and DBAs to manage and document the database.
- **Relationships in Relational Databases**
 - One-to-One, One-to-Many, Many-to-Many relationships.
 - Established using primary and foreign keys.
 - Ensure logical connections between related tables.